

Exercices Python — Jour 4 (partie 2) : listes, tuples, ensembles, dictionnaires et ranges

Exercices — Python Jour 4 (Partie 2)

Listes, tuples, ensembles, dictionnaires et ranges

Contexte pédagogique : ces exercices portent **exclusivement** sur les notions vues dans le cours « *Les types de données complexes* » (Jour 4, Partie 2) :

- **Listes** : indexation, slicing, `.append()`, `.insert()`, `.extend()`, `.remove()`, `.pop()`, `.index()`, `.count()`, `.sort()`, `.reverse()`, `.copy()`, `.clear()`, listes imbriquées ;
- **Tuples** : immuabilité, déballage (*unpacking*), `.count()`, `.index()`, conversions `list()` / `tuple()` ;
- **Ensembles (set)** : unicité, `.add()`, `.discard()`, `.remove()`, opérateurs `|` (union), `&` (intersection), `-` (différence), `^` (différence symétrique), `.issubset()` / `<=` ;
- **Dictionnaires (dict)** : création, accès direct et via `.get()`, affectation, `.items()`, `.keys()`, `.values()`, `.update()`, `.pop()`, `.setdefault()`, dictionnaires imbriqués ;
- **Ranges (range)** : génération, propriétés, test d'appartenance, conversions vers `list()`, `tuple()`, `set()`.
- Les notions déjà acquises restent autorisées : méthodes de chaînes de caractères (`split()`, `strip()`, `upper()`...), `if` / `elif` / `else`, `match` / `case`, opérateurs de comparaison et logiques, boucle `for`, `len()`, `int()`, `sorted()`, ainsi que le bloc `try` / `except` pour intercepter une `TypeError` sur un tuple (montré explicitement dans le cours).

Règle du jeu : interdiction d'utiliser des notions non vues (pas de fonctions `def`, pas de listes ou d'ensembles en compréhension, pas d'expressions régulières). Le but est de **choisir le bon type de données** pour chaque situation et de **manipuler ses méthodes**, pas d'aller chercher des raccourcis plus avancés qui seront vus plus tard dans la formation.

Consigne générale de rendu : pour chaque exercice, écrivez un script Python qui déclare vous-même les variables de test indiquées (pas besoin de `input()`), exécute le traitement demandé, et affiche des messages clairs avec `print()` / f-strings. Testez bien **toutes** les valeurs du jeu d'essai fourni : un bon script doit donner le bon résultat sur chacune d'entre elles.

Exercice 1**Gestion d'un inventaire de matériel réseau**

Contexte. Vous gérez, avec une simple liste Python, le stock de matériel réseau d'un magasin informatique. Le stock évolue en permanence : arrivées, ventes, retours, erreurs à corriger. Vous devez enchaîner plusieurs opérations classiques sur cette liste.

Jeu d'essai.

```
stock = ["cable-rj45", "switch-8ports", "routeur-wifi", "cable-rj45",  
         "point-acces", "switch-8ports", "cable-rj45"]
```

Consignes. Dans cet ordre précis :

1. Affichez le stock de départ, ainsi que son nombre total d'articles avec `len()`.
2. Un nouvel article "pare-feu" arrive : ajoutez-le à la **fin** du stock avec `.append()`.
3. Un client retourne un "switch-8ports" défectueux : retrouvez la position du **premier** "switch-8ports" avec `.index()`, puis insérez "switch-8ports-defectueux" juste après cette position avec `.insert()`.
4. Un lot de 3 nouveaux articles arrive en une seule livraison : ["baie-brassage", "onduleur", "cable-fibre"]. Ajoutez-les tous d'un coup à la fin du stock avec `.extend()`.
5. Le magasin vend un "cable-rj45" : retirez-en **un seul exemplaire** (la première occurrence) avec `.remove()`.
6. Affichez combien de "cable-rj45" il reste en stock avec `.count()`.
7. Retirez et affichez le **dernier** article du stock avec `.pop()` (sans préciser d'indice).
8. Triez le stock avec `.sort()` et affichez-le.
9. Faites une copie indépendante du stock final avec `.copy()`, ajoutez un article "test-copie-uniquement" **uniquement** à cette copie, puis affichez le stock d'origine et la copie pour bien montrer qu'ils sont désormais différents.

Contraintes techniques obligatoires. `len()`, `.append()`, `.index()`, `.insert()`, `.extend()`, `.remove()`, `.count()`, `.pop()`, `.sort()`, `.copy()`.

Résultat attendu (exemple, vérifié à l'exécution).

```
Stock de depart (7 articles) : ['cable-rj45', 'switch-8ports', 'routeur-wifi',  
'cable-rj45', 'point-acces', 'switch-8ports', 'cable-rj45']
```

```
Nb cable-rj45 restants apres la vente : 2
```

```
Dernier article retire (pop) : cable-fibre
```

```
Stock trie :
```

```
['baie-brassage', 'cable-rj45', 'cable-rj45', 'onduleur', 'pare-feu',  
'point-acces', 'routeur-wifi', 'switch-8ports', 'switch-8ports',  
'switch-8ports-defectueux']
```

```
Stock original (inchange, 10 articles) : [...meme liste que ci-dessus...]
```

```
Copie modifiee (11 articles) : [...+ 'test-copie-uniquement' a la fin]
```

Bonus (facultatif). Utilisez `.reverse()` sur une **copie** du stock trié pour l'afficher en ordre

alphabétique inversé, sans modifier le stock trié original.

Exercice 2



Une table de règles NAT figée

Contexte. Une table de redirection de ports (NAT/port-forwarding) associe, pour chaque règle, un port externe, une adresse IP interne, un port interne et un protocole. Une fois qu'une règle de pare-feu est déployée, elle ne doit **plus être modifiable** par erreur ailleurs dans le programme : c'est une situation typique où le **tuple** est préférable à la liste.

Jeu d'essai. Chaque règle est un tuple (port_externe, ip_interne, port_interne, protocole) :

```
regles_nat = [  
    ("80", "192.168.1.10", "80", "TCP"),  
    ("443", "192.168.1.10", "443", "TCP"),  
    ("2222", "192.168.1.20", "22", "TCP"),  
    ("5353", "192.168.1.30", "53", "UDP"),  
]
```

Consignes.

1. Affichez le nombre total de règles avec `len()`.
2. Parcourez la liste avec une boucle `for` et le **déballage** (port_ext, ip_int, port_int, proto), et affichez chaque règle au format "Externe:<port_ext>/<proto> -> <ip_int>:<port_int>".
3. Recherchez, par comparaison manuelle dans une boucle `for` (**sans** fonction), la règle dont le `port_interne` vaut "22" (SSH), et affichez le tuple complet si trouvé, sinon un message clair.
4. Construisez une liste `protocoles = []` en y ajoutant (avec `.append()`) le protocole de chaque règle, puis utilisez `.count()` sur cette liste pour compter le nombre de règles "TCP".
5. Dans un bloc `try / except TypeError` (comme montré dans le cours), tentez de modifier le port externe de la première règle (`regles_nat[0][0] = "8080"`), et affichez le message d'erreur intercepté pour démontrer l'immuabilité du tuple.

Contraintes techniques obligatoires. tuple, déballage, `len()`, recherche manuelle par boucle `for`, `.append()` et `.count()` sur une liste, `try / except TypeError`.

Résultat attendu (exemple, vérifié à l'exécution).

```
Nb regles : 4  
Externe:80/TCP -> 192.168.1.10:80  
Externe:443/TCP -> 192.168.1.10:443  
Externe:2222/TCP -> 192.168.1.20:22  
Externe:5353/UDP -> 192.168.1.30:53  
  
Regle SSH trouvee : ('2222', '192.168.1.20', '22', 'TCP')  
Nb regles TCP : 3  
Immutabilite confirmee : 'tuple' object does not support item assignment
```

Bonus (facultatif). Puisqu'il est impossible de modifier un tuple directement, convertissez la première règle en liste avec `list(regles_nat[0])`, modifiez son port externe en "8080", puis reconvertissez-la en tuple avec `tuple(...)` et affichez le résultat :

```
Regle modifiee (bonus) : ('8080', '192.168.1.10', '80', 'TCP')
```

Exercice 3



Audit d'une migration de serveur par ensembles

Contexte. Un serveur vient d'être migré vers une nouvelle machine. Avant et après la migration, un scan Nmap a relevé les ports ouverts de chaque machine. Vous devez comparer les deux relevés à l'aide d'opérations ensemblistes pour documenter précisément ce qui a changé, et vérifier qu'aucun port dangereux n'est resté exposé.

Jeu d'essai.

```
ports_srv_a = {21, 22, 80, 443, 3389, 8080} # avant migration
ports_srv_b = {22, 80, 443, 8443, 9000}    # apres migration
ports_dangereux = {21, 23, 3389, 445}      # FTP, Telnet, RDP, SMB
```

Consignes.

1. Affichez le nombre de ports ouverts sur chaque serveur avec `len()`.
2. Calculez et affichez (triés avec `sorted()`) les ports **communs** aux deux serveurs (intersection `&`).
3. Calculez et affichez les ports **fermés** par la migration : présents sur A, absents sur B (différence `ports_srv_a - ports_srv_b`).
4. Calculez et affichez les **nouveaux** ports ouverts par la migration : présents sur B, absents sur A (différence `ports_srv_b - ports_srv_a`).
5. Calculez et affichez l'**union** de tous les ports historiquement ouverts sur l'un ou l'autre serveur.
6. Calculez et affichez la **différence symétrique** (`^`) : les ports dont l'état a changé entre les deux serveurs.
7. À l'aide de l'opérateur d'inclusion (`issubset()` ou `<=`), vérifiez que les ports « essentiels » {80, 443} sont bien ouverts **à la fois** sur le serveur A et sur le serveur B, et affichez un message de conformité pour chacun.
8. Pour chaque serveur, calculez l'intersection avec `ports_dangereux` : si le résultat est non vide, affichez un message d'alerte listant les ports dangereux exposés ; sinon, affichez un message rassurant.

Contraintes techniques obligatoires. `len()` sur un `set`, `&`, `-`, `|`, `^`, `issubset()`/`<=`, `sorted()`.

Résultat attendu (exemple, vérifié à l'exécution).

```
Nb ports A : 6    Nb ports B : 5
Ports communs      : [22, 80, 443]
Fermes apres migration : [21, 3389, 8080]
```

```

Nouveaux apres migration : [8443, 9000]
Union totale             : [21, 22, 80, 443, 3389, 8080, 8443, 9000]
Difference symetrique    : [21, 3389, 8080, 8443, 9000]

Ports essentiels <= A ? True
Ports essentiels <= B ? True

Risque serveur A : ports dangereux exposes -> {3389, 21}
Risque serveur B : OK, aucun port dangereux expose

```

Bonus (facultatif). Retirez le port 3389 de `ports_srv_b` avec `.discard()` (il n'y est déjà pas) et montrez, en affichant un message, que `.discard()` ne lève **aucune erreur** contrairement à `.remove()` sur un élément absent.

Exercice 4



Fiche de configuration d'un serveur (dictionnaire imbriqué)

Contexte. La configuration d'un serveur applicatif est représentée par un dictionnaire regroupant des informations générales et deux sous-dictionnaires imbriqués ("**reseau**" et "**stockage**"). Vous devez consulter, corriger et enrichir cette fiche sans jamais provoquer d'erreur lorsqu'une clé attendue est absente.

Jeu d'essai.

```

configuration = {
    "hostname": "srv-app-02",
    "reseau": {
        "ip": "192.168.5.20",
        "masque": "255.255.255.0",
        "passerelle": "192.168.5.1",
    },
    "stockage": {
        "disque_go": 512,
        "type": "SSD",
    },
    "actif": True,
}

```

Consignes.

1. Affichez le `hostname` par accès direct (`configuration["hostname"]`).
2. Affichez l'adresse IP en enchaînant les crochets : `configuration["reseau"]["ip"]`.
3. Utilisez `.get()` pour lire une clé "`cpu`" **absente** du premier niveau, avec la valeur par défaut "`non renseigné`", et affichez le résultat.
4. Utilisez `.get()` de façon imbriquée pour lire `configuration["stockage"].get("raid", "non configure")`, et affichez le résultat.
5. Modifiez la quantité de disque à 1024 par affectation directe sur le sous-dictionnaire "`stockage`".

6. Ajoutez une nouvelle clé "dns" avec la valeur "8.8.8.8" sous "reseau", par affectation directe.
7. Utilisez `.setdefault()` pour ajouter la clé "vlan" avec la valeur 10 au premier niveau (elle n'existe pas encore). **Rappelez ensuite** `.setdefault()` sur la **même** clé "vlan" avec une valeur différente (999), puis affichez la valeur finale de "vlan" pour vérifier qu'elle n'a **pas** été écrasée.
8. Utilisez `.update()` pour fusionner en une seule fois `{"environnement": "production", "actif": False}` dans le dictionnaire principal (la clé "actif" existe déjà et doit être écrasée).
9. Parcourez le sous-dictionnaire "reseau" avec `.items()` et affichez chaque paire au format " <cle> : <valeur>".
10. Supprimez la clé "masque" de "reseau" avec `.pop()`, en récupérant sa valeur dans une variable, puis affichez cette valeur récupérée ainsi que le sous-dictionnaire "reseau" final.

Contraintes techniques obligatoires. accès direct et imbriqué par crochets, `.get()` simple et imbriqué, affectation de clé (nouvelle et existante), `.setdefault()`, `.update()`, `.items()`, `.pop()`.

Résultat attendu (exemple, vérifié à l'exécution).

```

Hostname : srv-app-02
IP : 192.168.5.20
CPU (absent) : non renseigne
RAID (absent, imbrique) : non configure
VLAN apres double setdefault : 10

    ip : 192.168.5.20
    masque : 255.255.255.0
    passerelle : 192.168.5.1
    dns : 8.8.8.8

Masque retire : 255.255.255.0
Reseau final : {'ip': '192.168.5.20', 'passerelle': '192.168.5.1', 'dns': '8.8.8.8'}
```

Bonus (facultatif). Affichez la liste des clés de premier niveau restantes avec `list(configuration.keys())`, puis vérifiez avec l'opérateur `in` sur `configuration.values()` si la valeur "production" est bien présente parmi les valeurs de premier niveau.

Exercice 5



Scan d'une plage de ports et synthèse

Contexte. Un scanner de sécurité teste systématiquement une **plage** de ports d'une machine et signale séparément les ports qu'il a effectivement trouvés ouverts. Votre travail est de croiser la plage théoriquement scannée avec les ports réellement détectés ouverts, en repérant au passage une éventuelle anomalie (un port signalé ouvert alors qu'il n'a même pas été scanné).

Jeu d'essai.

```
port_debut = 20
port_fin   = 25
ports_ouverts_detectes = {21, 23, 26}  # 26 est volontairement hors plage
```

Consignes.

1. Créez `plage_scan = range(port_debut, port_fin + 1)` (n'oubliez pas que la borne de fin de `range()` est exclusive).
2. Affichez le nombre de ports couverts par ce scan avec `len(plage_scan)`, sans convertir `plage_scan` en liste.
3. Pour chacun des ports de test [19, 21, 25, 26], utilisez l'opérateur `in` directement sur `plage_scan` (toujours sans le convertir) pour afficher "Port <p> : dans la plage scannee" ou "Port <p> : hors de la plage scannee".
4. Convertissez `plage_scan` en un ensemble `ports_scannes = set(plage_scan)`.
5. Calculez, avec une intersection, les ports **réellement ouverts dans la plage scannée** (`ports_scannes & ports_ouverts_detectes`) et affichez le résultat trié.
6. Calculez, avec une différence, les ports détectés ouverts mais **situés hors de la plage scannée** (`ports_ouverts_detectes - ports_scannes`) : cela doit isoler le port 26. Affichez un message d'anomalie clair pour chacun de ces ports.
7. Construisez un dictionnaire `synthese = {}` en parcourant `plage_scan` avec une boucle `for` : pour chaque port, associez la valeur "ouvert" s'il appartient à `ports_ouverts_detectes`, sinon "ferme".
8. Parcourez `synthese` avec `.items()` et affichez chaque ligne au format "Port <port> : <etat>".

Contraintes techniques obligatoires. `range()`, `len()` sur un `range`, `in` sur un `range`, `set(range(...))`, `&`, `-`, `sorted()`, construction d'un dict dans une boucle `for`, `.items()`.

Résultat attendu (exemple, vérifié à l'exécution).

```
Plage scannee : 6 ports (de 20 a 25 inclus)

Port 19 : hors de la plage scannee
Port 21 : dans la plage scannee
Port 25 : dans la plage scannee
Port 26 : hors de la plage scannee

Ports reellement ouverts dans la plage : [21, 23]
Anomalie : port 26 detecte ouvert mais hors de la plage scannee !

Port 20 : ferme
Port 21 : ouvert
Port 22 : ferme
Port 23 : ouvert
Port 24 : ferme
Port 25 : ferme
```

Bonus (facultatif). `range()` se convertit aussi bien en liste, en tuple qu'en ensemble selon le besoin : affichez `tuple(plage_scan)` pour l'illustrer.

Critères de réussite (auto-évaluation)

Avant de considérer un exercice comme terminé, vérifiez que :

- votre script fonctionne sur **toutes** les valeurs du jeu d'essai fourni, y compris les cas « pièges » (bornes de **range**, valeurs hors plage, clés absentes, tentative de modification d'un tuple) ;
- vous n'utilisez **que** des notions vues dans le cours Jour 4 Partie 2 (et les notions déjà acquises en partie 1) : pas de **def**, pas de compréhensions de liste ou d'ensemble, pas d'expressions régulières ;
- chaque méthode ou structure listée dans les « contraintes techniques obligatoires » de l'exercice apparaît bien au moins une fois dans votre code ;
- vous avez choisi, pour chaque exercice, le type de données réellement le plus adapté (liste si les données évoluent, tuple si elles sont figées, set pour l'unicité et les comparaisons, dict pour associer une valeur à une clé, range pour générer une suite de nombres) ;
- les messages affichés sont clairs et correspondent exactement à ce qui est demandé.